

Sales Order Processing System

Problem Statement

A fashion retail company manages a large customer base along with a team of sales representatives through a Sales Order Processing System. The objective of this assignment is to work with relational database concepts, constraints, joins, filtering techniques, and SQL set operators using Salesman, Customer, and Orders tables.

Dataset Tables

1. Salesman Table

SalesmanId	SalesmanName	City	Commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13
5003	Lauson Hen	San Jose	0.12

2. Customer Table

CustomerId	CustomerName	City	Grade	SalesmanId
3002	Nick Rimando	New York	100	5001
3007	Brad Davis	New York	200	5001
3005	Graham Zusi	California	200	5002
3008	Julian Green	London	300	5002
3004	Fabian Johnson	Paris	300	5006
3009	Geoff Cameron	Berlin	100	5003
3003	Jozy Altidor	Moscow	200	5007
3001	Brad Guzan	London	NULL	5005

3. Orders Table

OrderNo	PurchaseAmount	OrderDate	CustomerId	SalesmanId
70001	150.5	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.5	2012-08-17	3009	5003
70007	948.5	2012-09-10	3005	5002
70005	2400.6	2012-07-27	3007	5001
70008	5760	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.4	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.6	2012-04-25	3002	5001

Tasks to Perform

Task 1

Insert a new record into the Orders table

Task 2

Apply the following constraints:

- Add a PRIMARY KEY constraint on the SalesmanId column in the Salesman table.
- Add a DEFAULT constraint on the City column in the Salesman table.
- Add a FOREIGN KEY constraint on the SalesmanId column in the Customer table.
- Add a NOT NULL constraint on the CustomerName column in the Customer table.

Task 3

Fetch records where:

- Customer name ends with the letter N
- Purchase amount is greater than 500

Task 4

Using SQL SET operators:

- Retrieve unique SalesmanId values from two tables.
- Retrieve SalesmanId values including duplicates from two tables.

Task 5

Display the following columns:

- Order Date
- Salesman Name
- Customer Name
- Commission
- City

Condition:

- Purchase amount should be between 500 and 1500

Task 6

Using RIGHT JOIN, fetch all records from:

- Salesman table
- Orders table

```
/** Working On Case study**/
```

```
create database Sql_casestudy1
select * from fact
select * from location
select * from product
```

Case study1

Q--- display the number of states present in the location table.

```
select count(distinct(state)) as total_state from location
```

Q---How many products are of regular type?

```
select count(*) as total_products from product where type = 'regular'
select count(distinct(product)) from product where type = 'regular'
```

Q---How much spending has been done on marketing of product ID 1?

```
Select sum(marketing) as total_marketing from fact where productid = '1'
```

Q---what is min sales of a product?

```
select min(sales) as Min_sales from fact
```

Q---Display the max cost of good sold(cogs)?

```
select max(cogs) as Max_cogs from fact
```

Q---Display the details of the product where product type is coffee.

```
Select * from product where product_type = 'coffee'
```

Q---Display the details where total_expenses are greater than 40.

```
select * from fact where total_expenses > 40
```

Q---What is the avg sales in area code =719

```
select avg(sales) as total_sales from fact where area_code = 719
```

Q--- Find out the total profit generated by colorado state.

```
select sum(profit) as total_profit from fact f
join location l on f.area_code =l.area_code
where l.state = 'colorado'
```

--- or different way

```
select * into total_generated from
( SELECT f.productid, l.area_code ,l.state,f.profit
FROM fact f
JOIN location l ON f.area_code = l.area_code
where l.state = 'colorado') fgt
```

```
select sum(profit) as total_profit from total_generated where state = 'colorado'
```

Q----Display the avg inventory for each product ID

```
select
productid,
avg(inventory) As avg_inventory from fact
group by
productid;
```

Q---Display state in a sequential order in a location table.

```
select distinct (state)
from location
order by state
```

Q---Display the average budget of the product where the average budget margin should be greater than 100. ↗

```
select productid,
avg(budget_margin) as Avg_bud_margin from fact
group by productid
having avg(budget_margin) > 100
```

Q---What is the total sales done on date 2010-01-01?

```
select
sum(sales) as total_sales
from fact
where date = '2010-01-01'
```

Q--- Display the avg total exp of each product id on an individual date.

```
select
productid,
date,
avg([Total_Expenses]) as avg_total_exp
from fact
group by
productid,
date
```

Q---Display the table with the following attributes such as date,product id,product type,product,sales,profit,state,area_code. ↗

```
select
f.date,f.productid,P.Product_Type,P.Product,f.sales,f.profit,l.state,l.area_code
from fact f
join location l on f.area_code = l.area_code
join product P on f.ProductId = p.ProductId ↗
```

Q16---Display the rank without any gap to show the sales wise rank.

```
select sales,
DENSE_RANK() over(order by sales desc) as rank_sales_wise from fact
```

or

```
select *,
DENSE_RANK() over(order by sales desc) as sales_rank_wise from fact
```

Q17---find the state wise profit and sales.

```
select * into state_sales_profit from
(select f.profit,f.sales,l.state
from fact f
join location l on f.area_code = l.area_code) c

select state,
sum(profit) as Total_profit,
sum(sales) as Total_sales
from state_sales_profit
```

```

group by
state;

----or different way

select state,
sum(profit) as total_profit,
sum(sales) as Total_sales
from fact f
join location l on f.area_code = l.area_code
group by
state

or
select l.state,
sum(f.profit) as total_profit,
sum(f.sales) as Total_sales
from fact f
join location l on f.area_code = l.area_code
group by l.state

```

Q- -- find the state wise profit and sales alongwith product name.

```

select
l.state,p.product,
sum(profit) as total_profit,
sum(sales) as total_sales
from fact f
join location l on f.area_code =l.area_code
join product p on f.productid = p.productid
group by
l.state,p.product

```

Q19.---if there is a increase in sales of 5%,calculate the increased sales.

```

select sales,
sales * 1.05 as increased_sales from fact
group by sales
---or

select sales * 1.05 as increased_sales from fact

```

Q20.----Find the maximum profit along with the productid and product type

```

select
[Product_Type],
ProductId,
max(profit) as max_profit
from fact f
join product p on f.ProductId = p.ProductId
group by

```

```

[Product_Type],
ProductId;-----note finding error in this as ambiguous productid name

or
select * into pps
from (select f.productid,p.product_type,f.profit
from fact f
join product p on f.ProductId = p.ProductId) c

select
[Product_Type],
ProductId,
max(profit) as max_profit
from pps
group by
[Product_Type],
ProductId;

```

Q22---create stored procedure to fetch the result according to the product type from product table. ➤

```

create procedure Product_type @prod_type varchar(20) as
select * from product
where product_type = @prod_type

exec Product_type @prod_type = 'coffee'

```

Q--- write a query by creating a condition in which if the total expenses is less than 60 then it is a profit or else loss. ➤

```

select
total_expenses,
case
when Total_Expenses < 60 then 'profit' else 'loss'
end as Profit_loss_status
from fact

or
select total_expenses, if(total_expenses <60, 'profit', 'loss') as result from
fact ➤

```

Q23--- Give the total weekly sales value with the date and product id details . Use roll-up to up to pull data in heirarchical order. ➤

```

select DATEPART(week , date) as week ,date
productid,
sum(sales) as Total_weekly_sales
from fact
group by ROLLUP (DATEPART(week, date ), date, productid);

```

Q24--- Apply union and intersection operator on the tables which consist of attribute area code.

```
select area_code from fact
union
select area_code from location

select area_code from fact
intersect
select area_code from location
```

Q25--- Create a user- defined function for the product table to fetch a particular product type upon the user's preference.

```
CREATE FUNCTION Prod_Type (@Prod_Type VARCHAR(50))
RETURNS TABLE
AS
RETURN
(
    SELECT * from product
    WHERE Product_Type = @Prod_Type
);
```

```
select * from Prod_Type('coffee')
```

----Need to discussion this topic for more classification

Q26---Change the prouct type from coffee to tea where product id is 1 and undo it.

```
select * from Prod_Type('tea')
```

```
Begin tran
UPDATE Product
SET Product_Type = 'tea'
WHERE ProductId = 1;
```

```
rollback
```

Q27----Display the date,product ud and sales where total expenses are between 100 and 200.

```
select
date,
productid,
sales
from fact
where Total_Expenses between 100 and 200
```

Q28---Delete the reords in the product table for regular type.

```
Delete from product where type ='regular'
```

Q29--Display the ASCII value of the fifth character from the column product.

```
select ASCII (substring (Product,5,1)) as ASCII_value from Product;  
select * from product
```

Now ,Lets export data to Power BI to create best meaning full insights for business

#Creating a column for total_sales and Total_profit

```
SELECT  
state,  
SUM(sales) AS total_sales,  
SUM(profit) AS total_profit  
FROM fact f  
JOIN location l  
ON f.area_code = l.area_code  
GROUP BY state
```

```
SELECT @@SERVERNAME;
```

#Weekly sales analysis

```
SELECT  
    DATEPART(WEEK, date) AS week_no,  
    SUM(sales) AS total_weekly_sales  
FROM fact  
GROUP BY DATEPART(WEEK, date)  
ORDER BY week_no;
```

Rank function sales rank by total sales

```
SELECT  
    p.product,  
    SUM(f.sales) AS total_sales,  
    DENSE_RANK() OVER(  
        ORDER BY SUM(f.sales) DESC  
    ) AS sales_rank  
FROM fact f  
JOIN product p  
ON f.productid = p.productid  
GROUP BY p.product;
```

Ranks state by profit

```
SELECT  
    l.state,  
    SUM(f.profit) AS total_profit,
```



```
DENSE_RANK() OVER(  
    ORDER BY SUM(f.profit) DESC  
    ) AS profit_rank  
FROM fact f  
JOIN location l  
ON f.area_code = l.area_code  
GROUP BY l.state;
```

 Auto recovery contains some recovered files that haven't been opened.

[View recovered files](#)



Business decision dashboard

(Sales Order Processing System Analysis)

January	February	March	April
May	June	July	August
September	October	November	December

- ☐ California
- ☐ Colorado
- ☐ Connecticut

Sum of Sales

820K

Sum of Profit

260K

Sum of Total_Expenses

230K

Sum of Marketing

132K

Sum of Margin

443K

Sum of Sales and Sum of Profit by Total_Expenses

412✓

Goal: 49 (+740.82%)
190

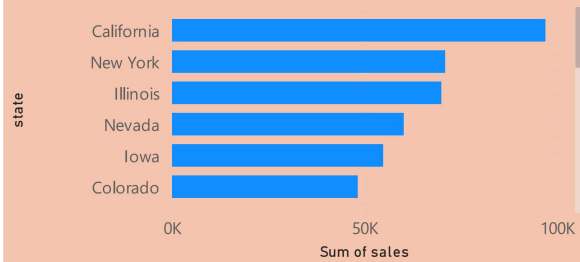
First state

Colorado

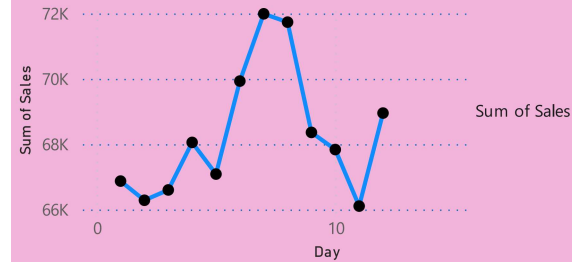
Sum of profit

18K

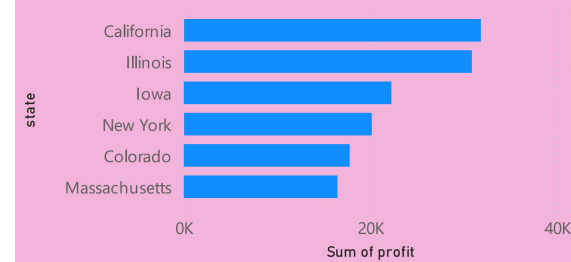
Sum of sales by state



Sum of Sales by Day



Sum of profit by state



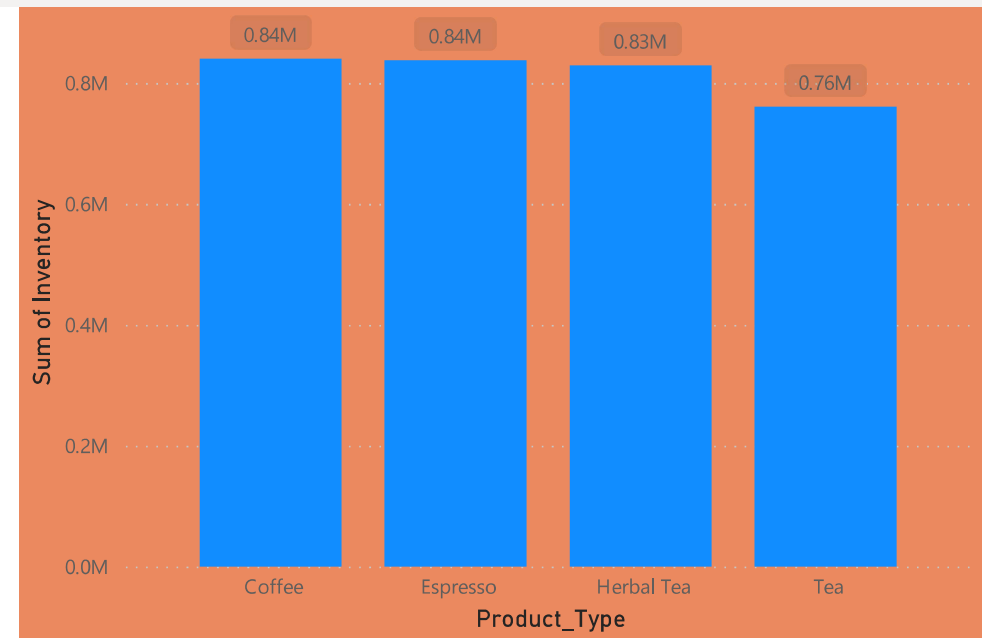
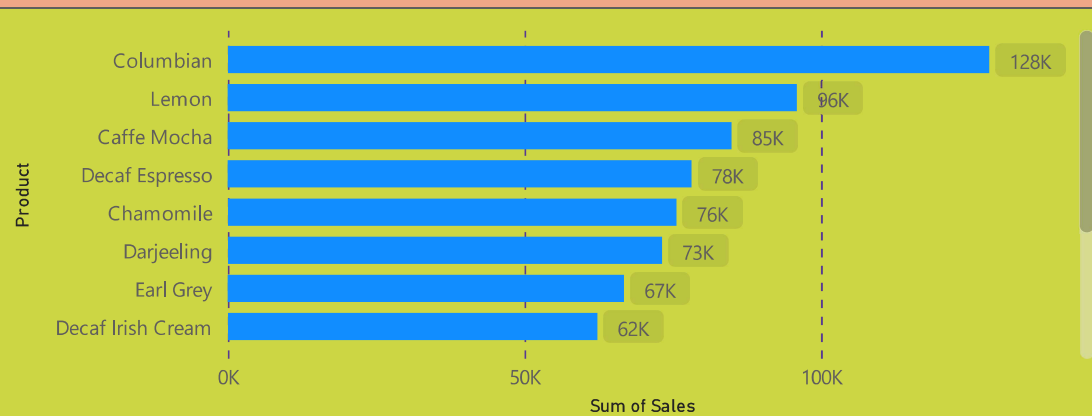
i Auto recovery contains some recovered files that haven't been opened.

View recovered files

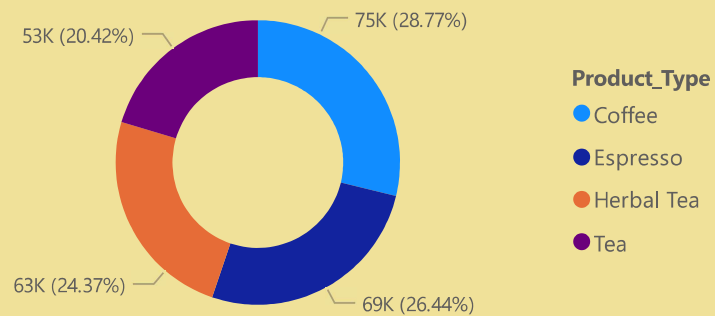


1	2	3	Herbal Tea	1972	New Hampshire
4	5	6	Tea	2267	New Hampshire
7	8	9	Espresso	2313	Iowa
			Herbal Tea	2892	New Mexico
			Total	819811	

Sum of Sales by Product



Sum of Profit by Product_Type

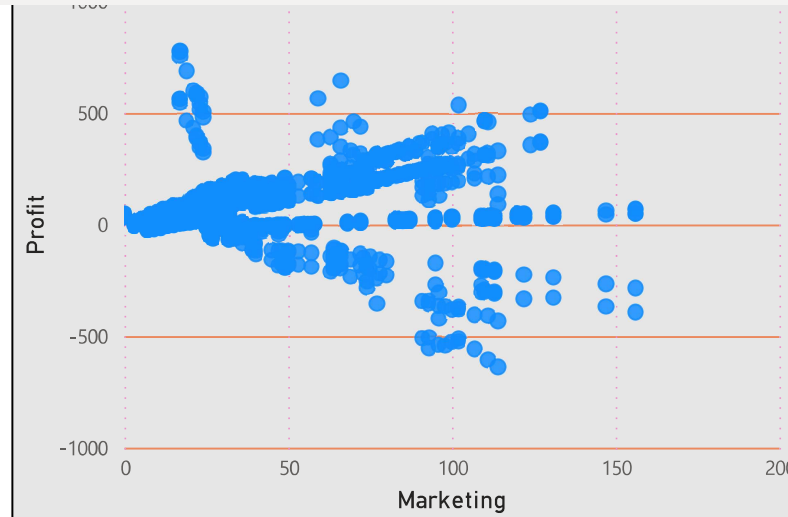
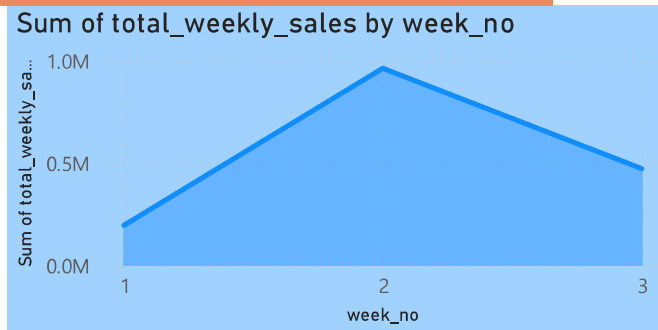


Product Performance Analysis Insights

- Product-level analysis helped identify high-performing and low-performing products based on sales and profitability metrics.
- Coffee category products generated a significant share of total sales contribution across multiple regions.
- Inventory analysis highlighted products with higher stock levels but comparatively lower sales performance, indicating possible inventory optimization opportunities.
- DENSE_RANK() window function was used to rank products based on total sales contribution and identify top-performing products.
- Product-wise profitability analysis provided insights into which product categories delivered better operational margins.

Auto recovery contains some recovered files that haven't been opened.

--	Coffee	Espresso
Herbal Tea	Tea	



View recovered files X

California	31785	1
Colorado	17743	5
Connecticut	7621	16
Florida	12310	9
Illinois	30821	2
Iowa	22212	3
Louisiana	7355	17
Massachusetts	16442	6
Missouri	3601	18
Nevada	10616	12
New Hampshire	2748	19
New Mexico	799	20
New York	20096	4
Total	259543	210

Business Insights

Key Insights:

- Weekly sales analysis identified strong sales concentration during specific operational periods.
- Coffee products generated the highest sales contribution across regions.
- States with high sales did not always generate proportional profit, indicating cost optimization opportunities.
- Marketing expenditure showed positive relationship with profitability in several cases.
- DENSE_RANK() analysis helped identify top-performing products and business segments.